



POLITECNICO MILANO 1863

Design Document

Ali Noshad, ali.noshad@mail.polimi.it, 101108

November 27, 2024

Students&Companies - Design Document
Prof. Matteo Rossi

Contents

1	Introduction	3
1.1	Purpose	3
1.2	Scope	3
1.3	Definition, Acronyms, Abbreviations	3
1.4	Revision History	3
1.5	Reference Documents	3
1.6	Document Structure	3
2	Architectural Design	4
2.1	Overview	4
2.2	Component View	5
2.3	Deployment View	9
2.4	Runtime View	11
2.4.1	Registering as Student	11
2.4.2	Registering as Company	12
2.4.3	Registering as University	13
2.4.4	Login	14
2.4.5	Manage Resume	15
2.4.6	View Internship	16
2.4.7	Sending Application	17
2.4.8	Track Application	18
2.4.9	Complaint Submission	19
2.4.10	Questionnaire Submission	20
2.4.11	Accept/Reject Offer	21
2.4.12	Meeting Attendance	22
2.4.13	Internship Posting	23
2.4.14	View Applicants	24
2.4.15	Shortlist Applicants	25
2.4.16	Schedule Meeting	26
2.4.17	Manage Questionnaire	27
2.4.18	Send Offer	28
2.4.19	Update Applicant Status	29
2.4.20	Monitor Internships	30
2.5	Component Interfaces	30
2.6	Selected Architectural Styles and Patterns	33
2.7	Other Design Decisions	34
2.7.1	Availability	34
2.7.2	Scalability	34
2.7.3	Notification Handling	34
2.7.4	Ease of Deployment	34
2.7.5	Data Storage	34
3	User Interface Design	35
3.1	Overview	35
3.2	Header	35
3.3	User Interfaces	35
4	Requirements Traceability	35
5	Implementation, Integration and Test Plan	35
6	Effort Spent	35

1 Introduction

1.1 Purpose

This document aim to provide technical information of S&C platform. This document aims to assist the development team in implementing the desired system. The document describes the system architecture and its components in detail, including interactions between them. The Design Document (DD) defines implementation, integration, and testing strategies based on priority, effort, and stakeholder impact. In general, the main different features of this document are:

- The high-level architecture
- Main components of the system
- Interfaces provided by the components
- Design patterns adopted
- Implementation, integration, and testing

1.2 Scope

Students&Companies (S&C) is a new platform that helps students to find internships, by browsing through the platform, in which they search for an internship that a company has posted, and sending their resume and initiate their application. Companies post internships allowing students to apply for those internships. Furthermore, universities monitor the application/internship of their students, and allowing students to report a complaint regarding any stage of the procedure.

1.3 Definition, Acronyms, Abbreviations

Table 1: Abbreviations	
Abbreviations	Description

1.4 Revision History

1.5 Reference Documents

- Specification Document: “Assignment RDD AY 2024-2025.pdf”
- Course Slides

1.6 Document Structure

Section 1 Brief description of DD and introduction of purpose and scope. It also includes definitions, acronyms, and abbreviations.

Section 2 The main part of DD is this section that contains architectural design choices, including all the components, and the interfaces used for the development of the application. Also, it contains a deployment view and a runtime view. In the end, is explained the architectural patterns chosen with the other design decisions.

Section 3 Contains how should be the user interface on mobile applications.

Section 4 Contains the traceability matrix that shows which components satisfy certain requirements.

Section 5 The implementation plan, integration plan, and testing plan show how these plans are executed.

Section 6 Shows how much time is spent by each member of the group.

Section 7 Includes the reference documents.

2 Architectural Design

2.1 Overview

The architecture of this platform is structured by 3 logic layers:

1. **Presentation Layer:** The presentation Layer provides means of Input, allowing the users to manipulate the system Output, allowing the system to produce the results of user manipulation. SAP is having Graphical User interface (SAP GUI). The SAP GUI is installed on Individual machines which act as a presentation layer.
2. **Application Layer:** In this layer business logic is executed. The application layer can be installed on one machine, or it can be distributed among more than one system.
3. **Database Layer:** The database layer holds the data. SAP supports any relational database. SAP does not provide any database. But it supports any RDBMS. The database layer must be installed on one machine or system. Major databases which are being used in SAP implementations are Oracle and DB2.

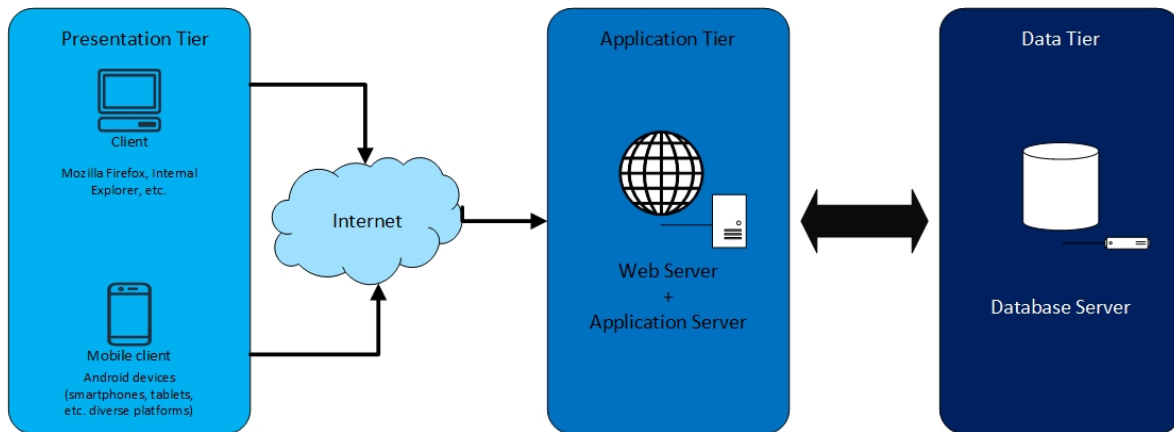


Figure 1: Three tier architecture

The following figure represents our system architecture, its high-level components, and their interaction. The client could use this by a web browser on their smartphone or computer. Then, user interactions will be processed by the application server. As well as an application server connected to the database. Finally, the application server is allowed to use APIs for Google Meets, Google Calendar.

These figures do not include all details about the architecture of the application, and in the next section, more details are represented.

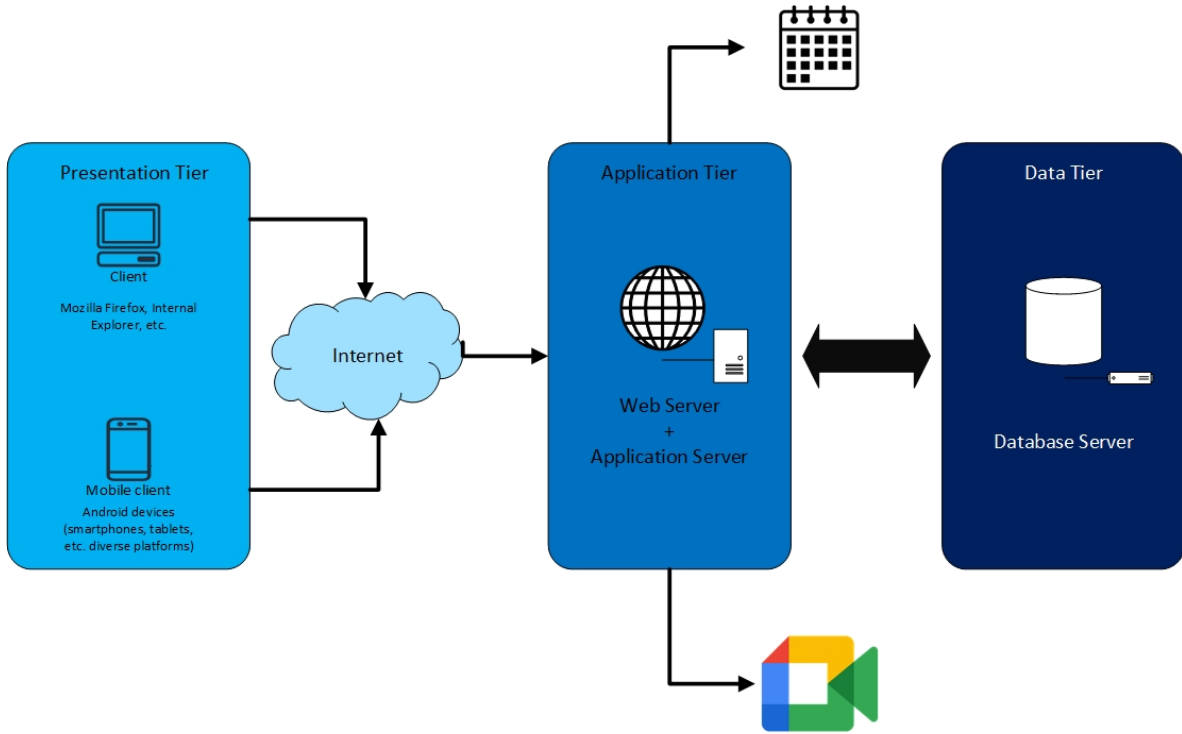


Figure 2: System Architecture

2.2 Component View

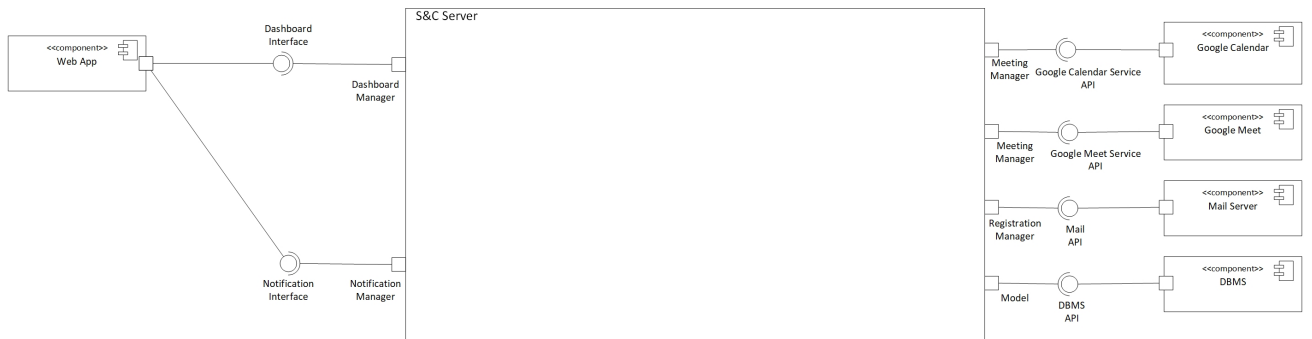


Figure 3: High Level Component Diagram

The high level component diagram above depicts S&C's external components and their communication with the server.

Web Server: The Dashboard Interface is the only way for users to communicate with the S&C Server. The S&C Server may send notifications to users via the Notification Interface, including meeting request, internship offer and etc.

DBMS: The DBMS stores all User, applications, complaints, and internships. The Model component connects to the S&C Server via the DBMS API.

Mail Server: In charge of sending emails confirming registration, the Mail Server uses the Mail API interface to connect to the S&C Server. The Registration Manager component, which manages

the user registration procedure, is connected to this interface.

Google Meet: The Google Meet service is used for holding meetings in between students and company's representative for internship positions. The Google Meet service uses the Google Meet Service API interface to connect to the S&C Server. The Meeting Manager component, which manages the meetings between users, is connect to this interface.

Google Calendar: The Google Calendar service is used for keeping information about the availability regarding the meetings and internships. The Google Calendar service uses the Google Calendar API interface to connect to the S&C server. The Meeting Manager component, which manages the meetings between users, is connect to this interface.

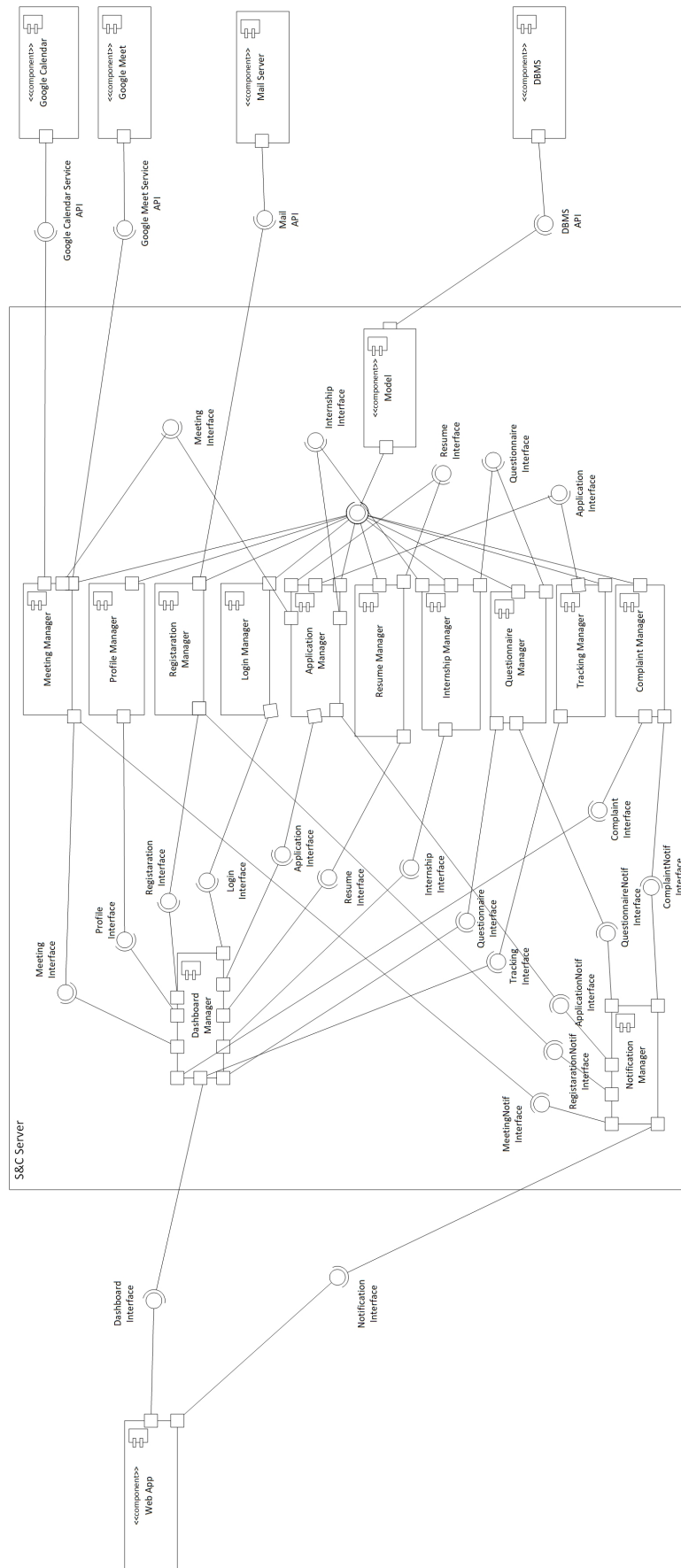


Figure 4: Low Level Component Diagram

The figure above depicts the whole architecture of the S&C website, including each component inside the S&C Server:

Dashboard Manager is a key component that facilitates communication between users and the S&C website. The Dashboard Interface allows users to interact with S&C, while the Dashboard Manager distributes requests to the necessary components. It acts as the primary center for user interactions.

The **Model** component contains an object-relation mapping (ORM) that other components use when retrieving data from the database. Thus, it is the component solely responsible for communicating with the data tier.

The **Registration Manager** is the component that manages new user registration. To create an account on the S&C system, a new user contacts with the Dashboard Manager, who then transmits the request to the Registration Manager via the registration interface. The Registration Manager sends a confirmation email to the new user and adds their information to the database using the Mail API and Model Interface. This component allows registered companies to manage internship and allows the registered students to look for internship and imitate the recruitment process.

Login Manager A component that manages the login procedure for registered users. When a user attempts to log in, the Dashboard Manager sends a request to the Login Manager via the Login Interface. The Login Manager uses the Model interface to obtain User data from the DBMS.

Notification Manager is component that handles each notification that has to be sent to the user, for example, when a new meeting is requested for an student, when user registers, any update regarding the application process for an internship, when a new questionnaire created by the a company use it send notification to all student applicants that applied for that specific internship and alerting about the new assignment, and finally we a new complaint has been submitted by an student regarding the application or internship process, a notification is sent to the university.

Application Manager handles all the users action regarding the an on-going application for an internship. It communicates with the Dashboard Manager through the Application Interface, with the Notification Manager through the ApplicationNotif Interface when a new Application is created or closed and with the Model component through the Model Interface to add or retrieve data from the DBMS.

The **Profile Manager** is responsible for managing user-specific data, including the storage, retrieval, and updating of profiles for students, companies, and universities. It provides the ability for each user type to access and modify their respective data. For example, students can update personal details or their academic credentials, while companies can update organization information or manage recruiter profiles. This component interacts with the Profile Interface to gather user inputs and provide real-time updates. It also serves as a central source of user data for other components, such as the Internship Manager, which links student profiles to internship applications, or the Login Manager, which validates user credentials.

The **Internship Manager** handles the creation, management, and tracking of internships. The main function of this component is to allow companies to post internships and manager their internship details, such as requirements, application deadlines, and enabling students to search, apply for, and track their application. This component communicates with Internship Interface, in which companies input internship details and students view available opportunities. The Internship Manager also communicates with other components such Application Manager to manage application, and Notification Manager to notify students about application updates and companies about the students actions on applications.

The **Resume Manager** is responsible for creation, storage, and management of student resumes, acting as a centralized system for handling resume related tasks. An student can upload an already

built resume, or use a functionalities in the system to generate resumes directly via Resume Interface. This component ensures that resumes are stored securely and can be retrieved for various purposes, such as applying for internships and sharing with hiring managers. It provides interface for Application Manager, which pulls the resume during internship application process.

The **Questionnaire Manager** is responsible for creation, assignment, and evaluation of the questionnaires within the system, which functions for both companies and students. Companies can use this component to design questionnaire with goal of assessing applicant's skills, knowledge, or personality as part of application procedure. Through the Questionnaire Interface, companies can create custom questionnaires and students can access and complete them. It also provides interface with internship manager, linking questionnaire with a specific internship, and the Notification Manager, which notifies users about assignment of questionnaire or submission of a questionnaire.

The **Tracking Manager** is responsible for monitoring and managing the status and progress of internship applications and related activities for students. Its main function is to provide real-time updates on application statuses, such as submission, review, shortlisting, or rejection, ensuring transparency for all parties involved, particularly university users. Through the Tracking Interface, universities can view the progress of their student. The Tracking Manager interacts with Application Manager to retrieve application data and updates.

The **Complaint Manager** is responsible for submission, reception of complaints within the system, and enables effective communication and problem reporting among students and universities. Through the Complaint Interface, users can submit complaint regarding any issues during the application and internship procedure. Once submitted, the Complaint Manager categorize and records complaint and ensures that they are directed to appropriate university. It interacts with Notification Manager to keep users informed about the status of their complaint.

2.3 Deployment View

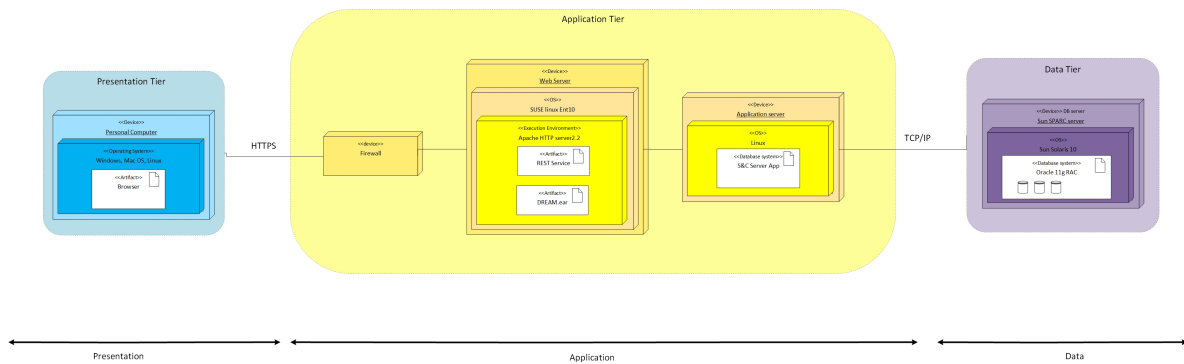


Figure 5: Deployment View Diagram

Personal Computer: Users can use their preferred web browser to access the system on any kind of personal computer. The Web Server and the browser will exchange information. Additionally, users can utilize any device that has a web browser, including tablets, smartphones, and more.

Web Server: All users that visit the system via a web browser can access the Application Server's service with the Web Server. Specifically, to manage high user traffic, the Web Server performs load balancing on client requests sent to the different Application Servers without executing any business logic. Additionally, it gives the client's browsers the HTML, JSON, Javascript, and CSS files so that the sites can be rendered.

Firewall: A firewall is used for providing safe access to the server and database, as well as, preventing external attacks.

Application Server: The system's business logic is housed in the application server. Additionally, it uses the Web Server-managed HTTPS protocol to connect with the client. The Dashboard Manager is responsible for directing the different requests that originate from the Web Server to the appropriate module. Additionally, it uses the model gateway to interface with the Database Server. To manage high user traffic, this node is duplicated.

Database Management System (DBMS): For performing the job of DBMS Oracle Real Application Cluster (RAC) has been selected. A cluster comprises multiple interconnected computers or servers that appear as if they are one server to the end-users and applications. Oracle RAC enables you to cluster Oracle databases and it is a configuration of Relation DBMS (RDBMS). Some nodes contain the same instance of the database and the instances are connected to each other and share the cache in the Global Cache (GC). Then, there is a pool of replicated databases that contain data files. Datafiles are common, shared and they are accessed from all instances in a parallel and synchrony way.

2.4.1 Registering as Student

2.4.1 Registering as Student



The diagram above illustrates the **Student Registration** process. The student accesses the Homepage of the platform via a web browser and is redirected to the Login page. After selecting the "Create Account" option, they are presented with a registration form. The student fills in their details, such as name, email, password, and any additional academic information, and ticks the "Register as a Student" option. The completed form is forwarded to the DashboardManager, which passes the data to the RegistrationManager. The RegistrationManager performs validation checks on the submitted information, ensuring no duplicate records exist and all fields are properly formatted. If the checks

pass, a confirmation email is sent to the student via the Mail Server. Once the student clicks the link in the confirmation email, the RegistrationManager saves their profile in the database through the Model component. The process completes by redirecting the student to their Dashboard.

2.4.2 Registering as Company

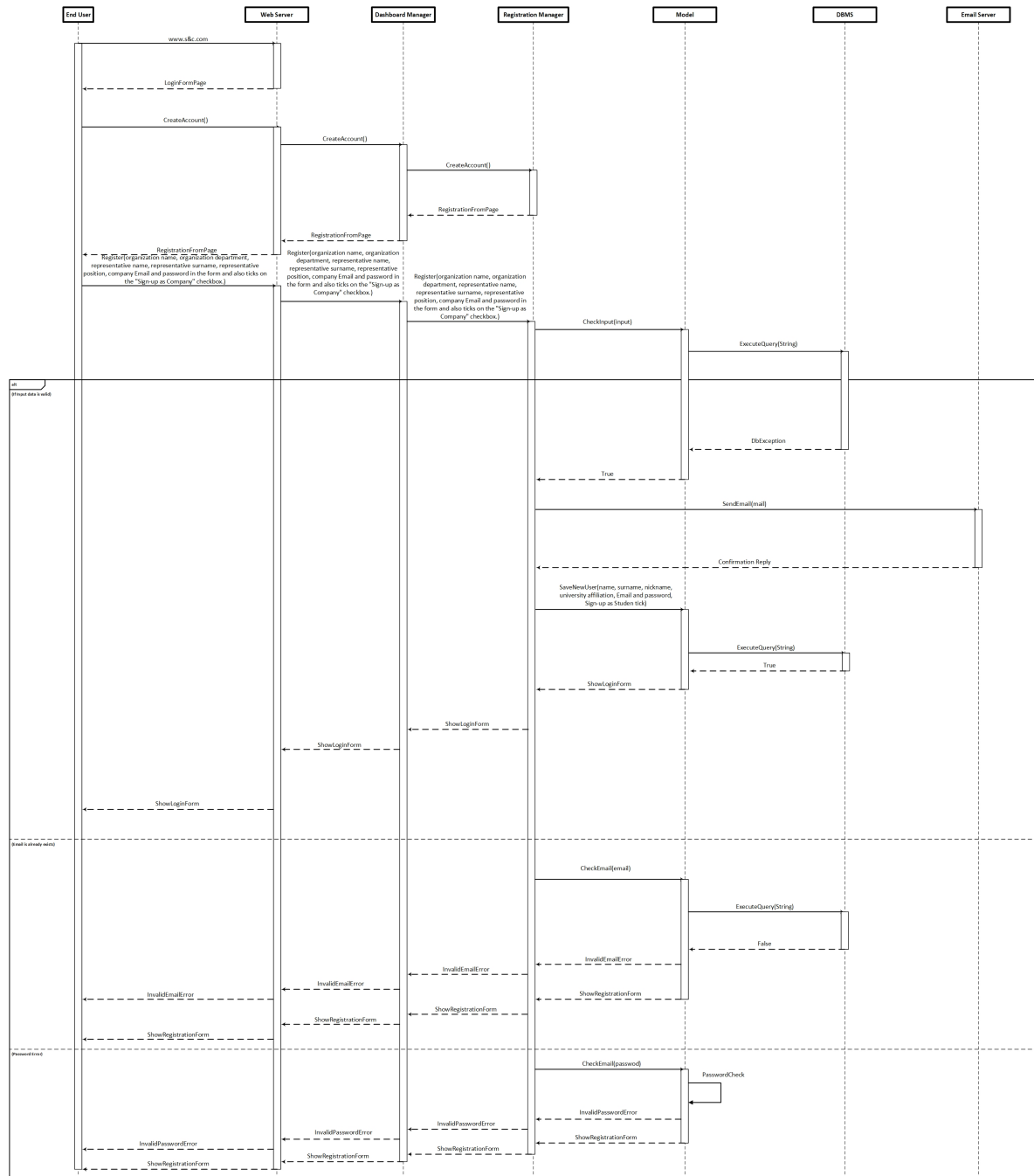


Figure 7: Company Registration Runtime View

This diagram illustrates the Company Registration process. A company representative starts by accessing the platform's landing page and navigating to the Login page. After selecting "Create Account" the representative is directed to the company registration form. They enter the company name, contact

information, and authentication credentials and mark the "Register as a Company" checkbox. The registration data is sent to the DashboardManager, which forwards it to the RegistrationManager. The system validates the data, checking for company uniqueness and proper formatting. If the details are valid, the Mail Server sends a verification email. Once the email is confirmed, the RegistrationManager adds the company's profile to the database via the Model interface. The representative is then redirected to the company's Dashboard.

2.4.3 Registering as University

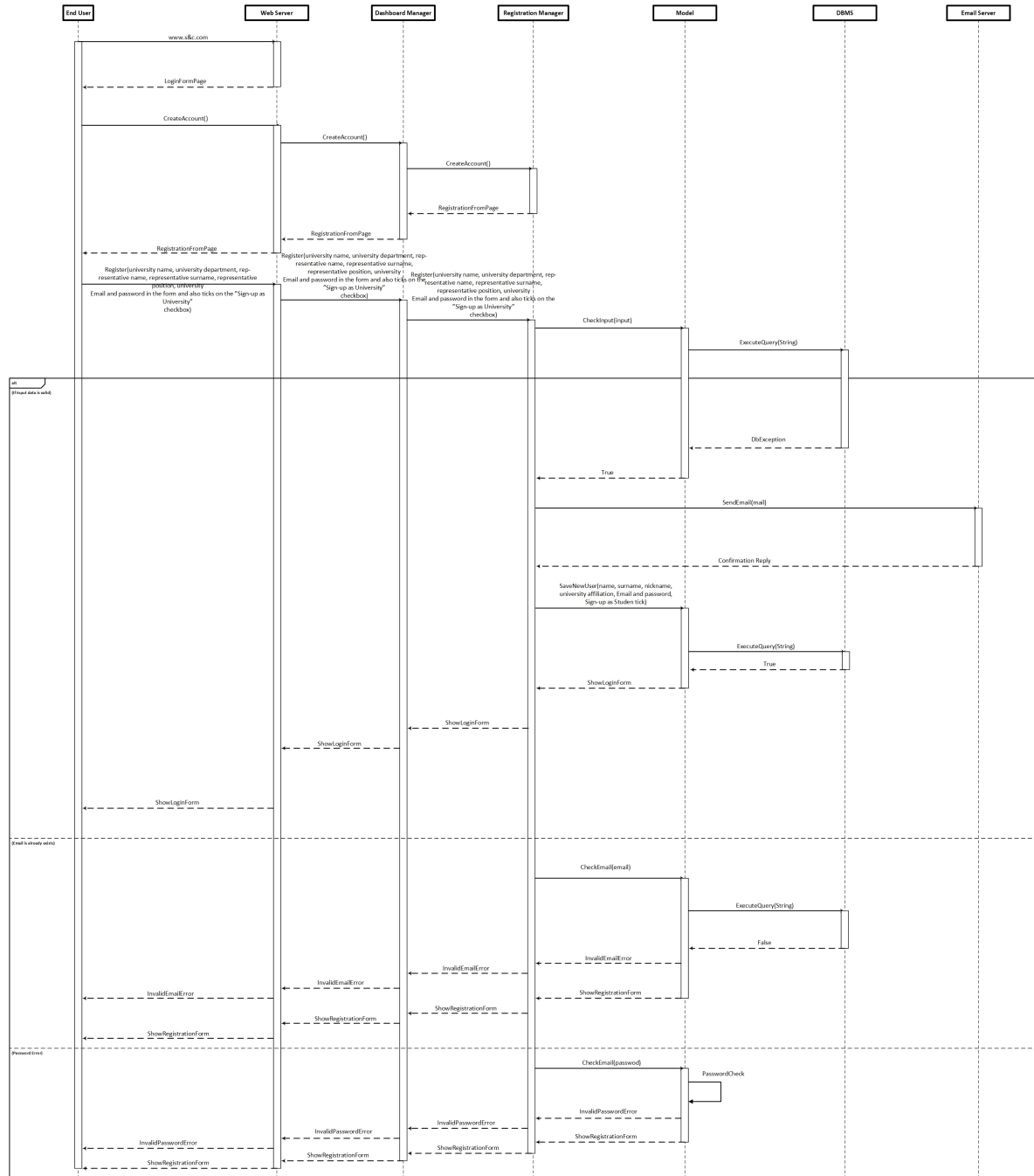


Figure 8: University Registration Runtime View

This diagram represents the University Registration process. The university representative accesses the platform and navigates to the Login page. After clicking on the "Create Account" button, they are redirected to the registration form. They provide institutional information, including university name, email, and address, and tick the "Register as a University" option. The completed form is sent to the DashboardManager, which passes the request to the RegistrationManager. The system performs validation checks on the data, ensuring proper format and non-duplication. If valid, a confirmation email is sent through the Mail Server. After clicking the email verification link, the university profile is saved in the database using the Model component, and the representative is redirected to the University Dashboard.

2.4.4 Login

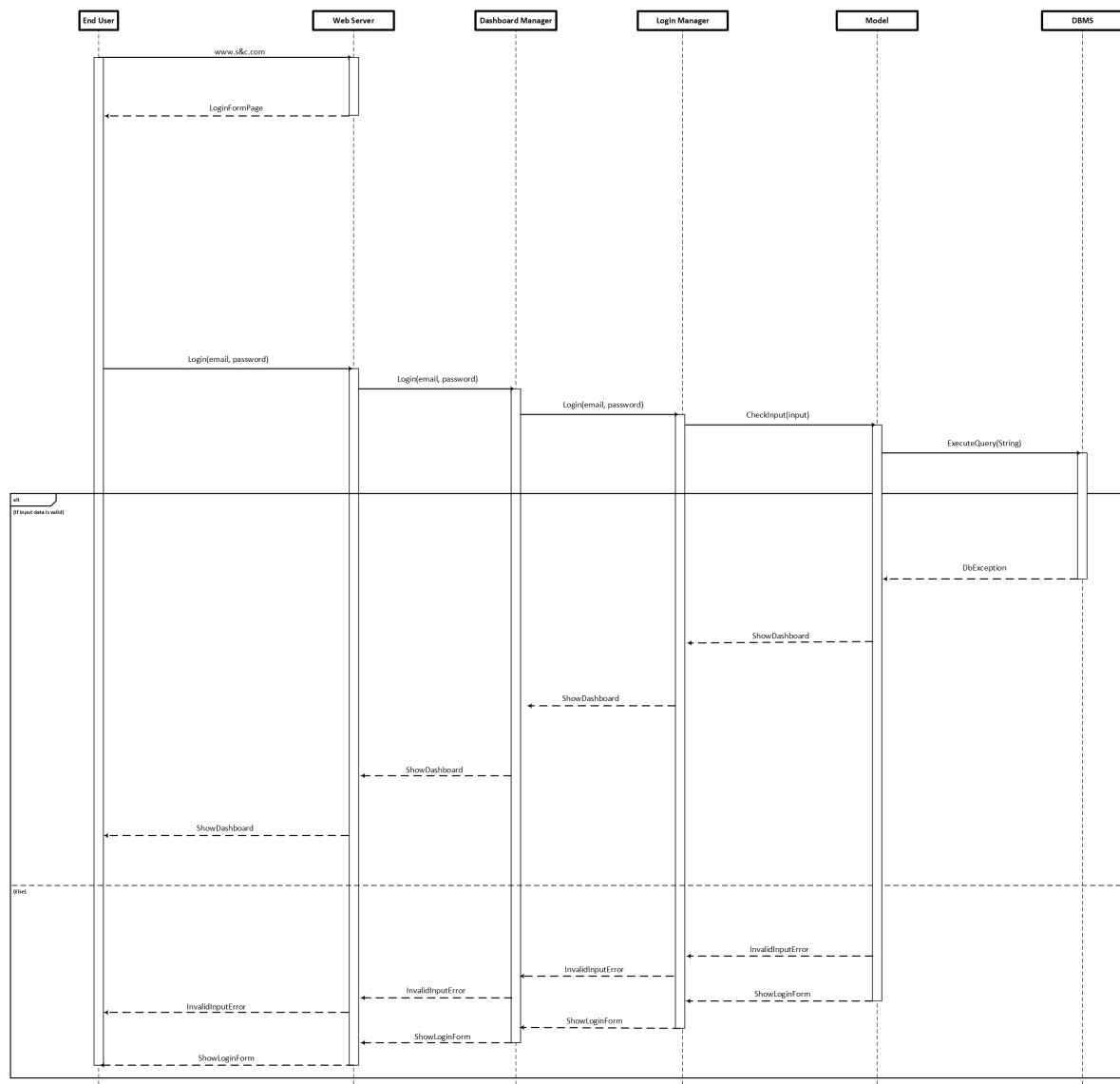


Figure 9: Login Runtime View

This diagram depicts the Login Process. Users (students, companies, or universities) navigate to the Login page of the platform. They provide their email and password, and the data is submitted to the DashboardManager. The request is forwarded to the LoginManager, which queries the database via the Model interface to verify credentials. If the credentials are correct, the LoginManager creates a

2.4.6 View Internship

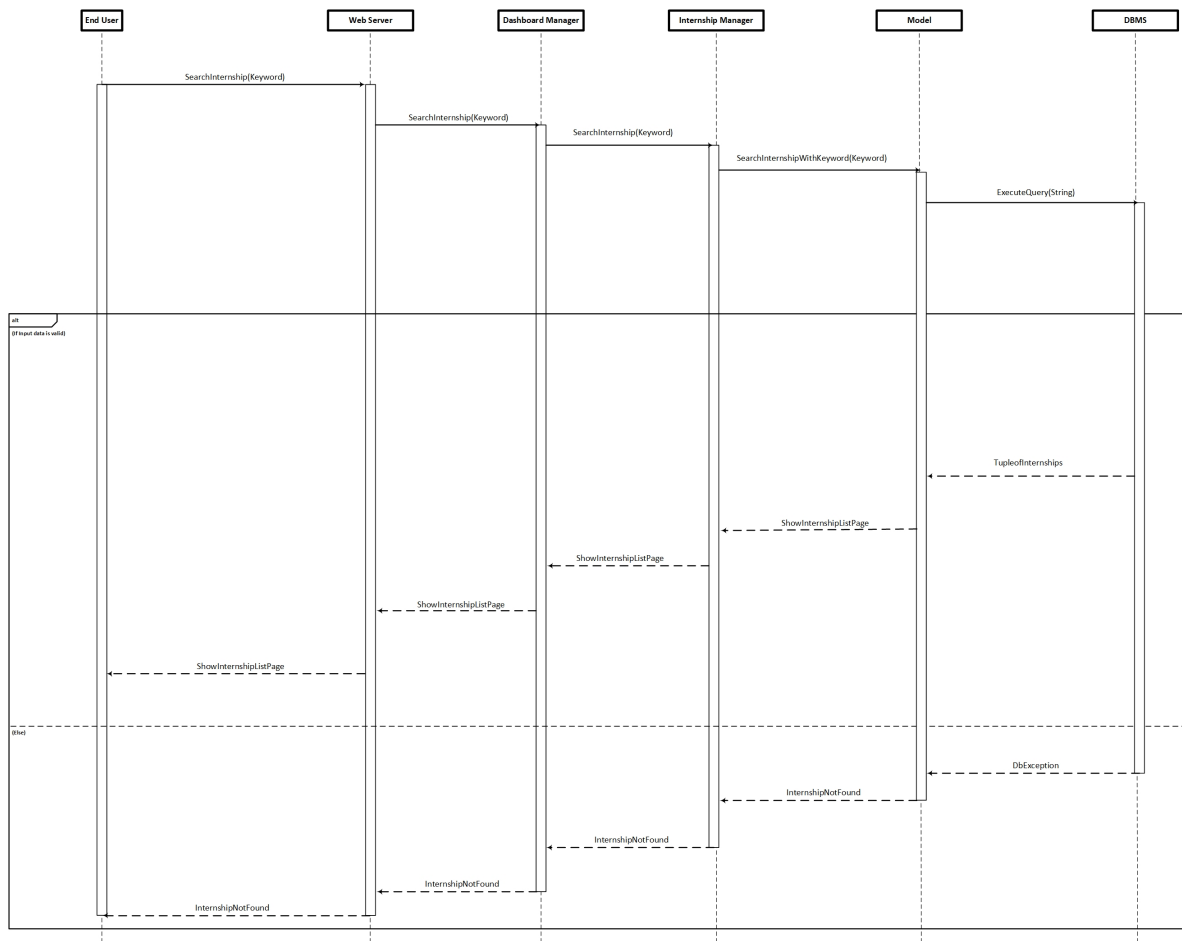


Figure 11: View Internship Runtime View

This diagram explains the Internship Browsing process. A student logs into their dashboard and accesses the internship search feature. They specify filters such as location, company, or type of internship. The DashboardManager sends the query to the InternshipManager, which retrieves relevant internships from the database via the Model interface. The results are displayed on the student's dashboard, allowing them to browse and select internships of interest.

2.4.7 Sending Application

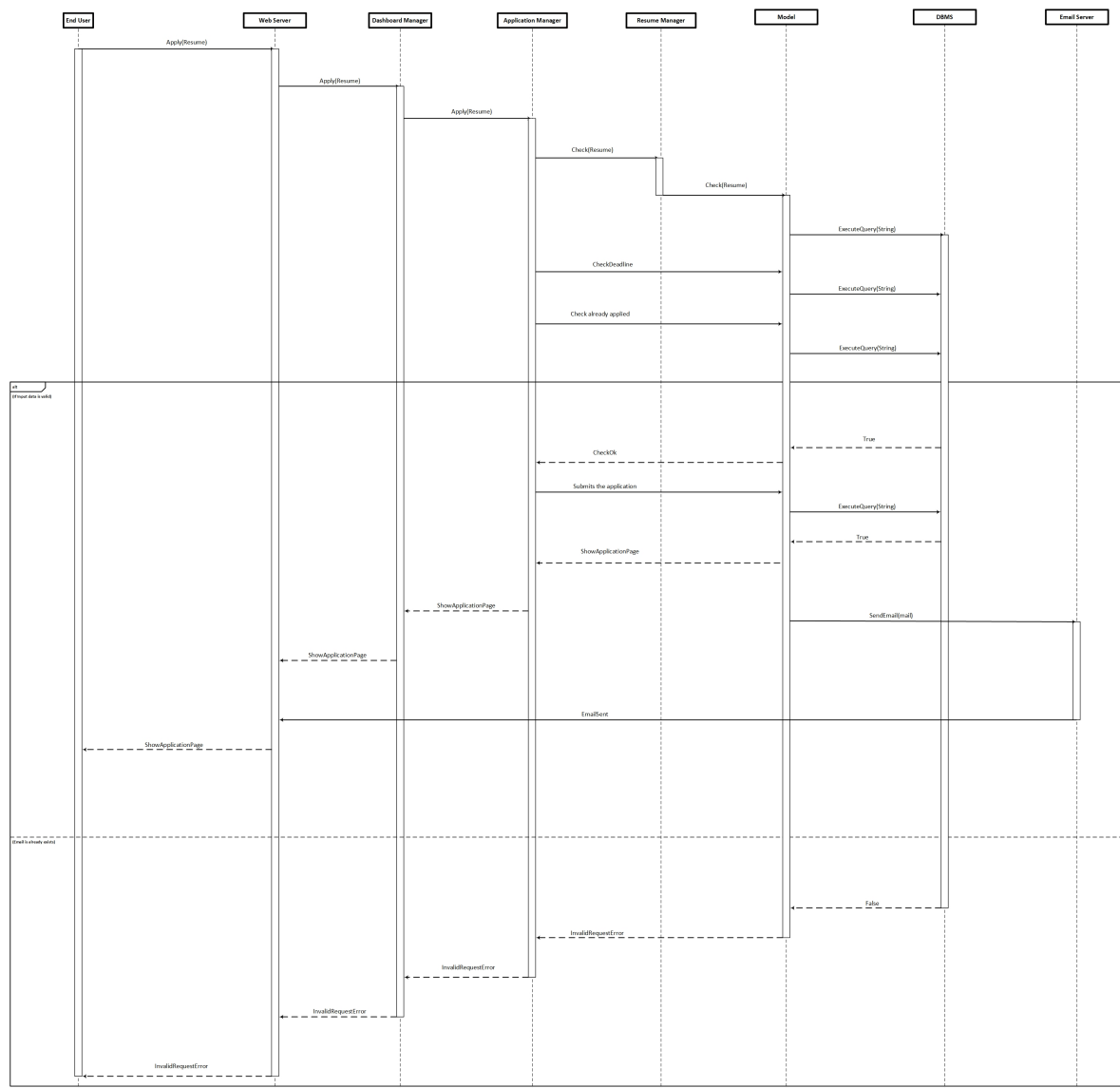


Figure 12: Sending Application Runtime View

This diagram details the Internship Application process. A student selects an internship from the list of available opportunities. The application, including their resume, is submitted via the Dashboard-Manager to the ApplicationManager. The ApplicationManager stores the application details in the database through the Model interface. A notification confirming the application is sent to the student and the company using the NotificationManager. The process concludes by displaying a confirmation message to the student.

2.4.8 Track Application

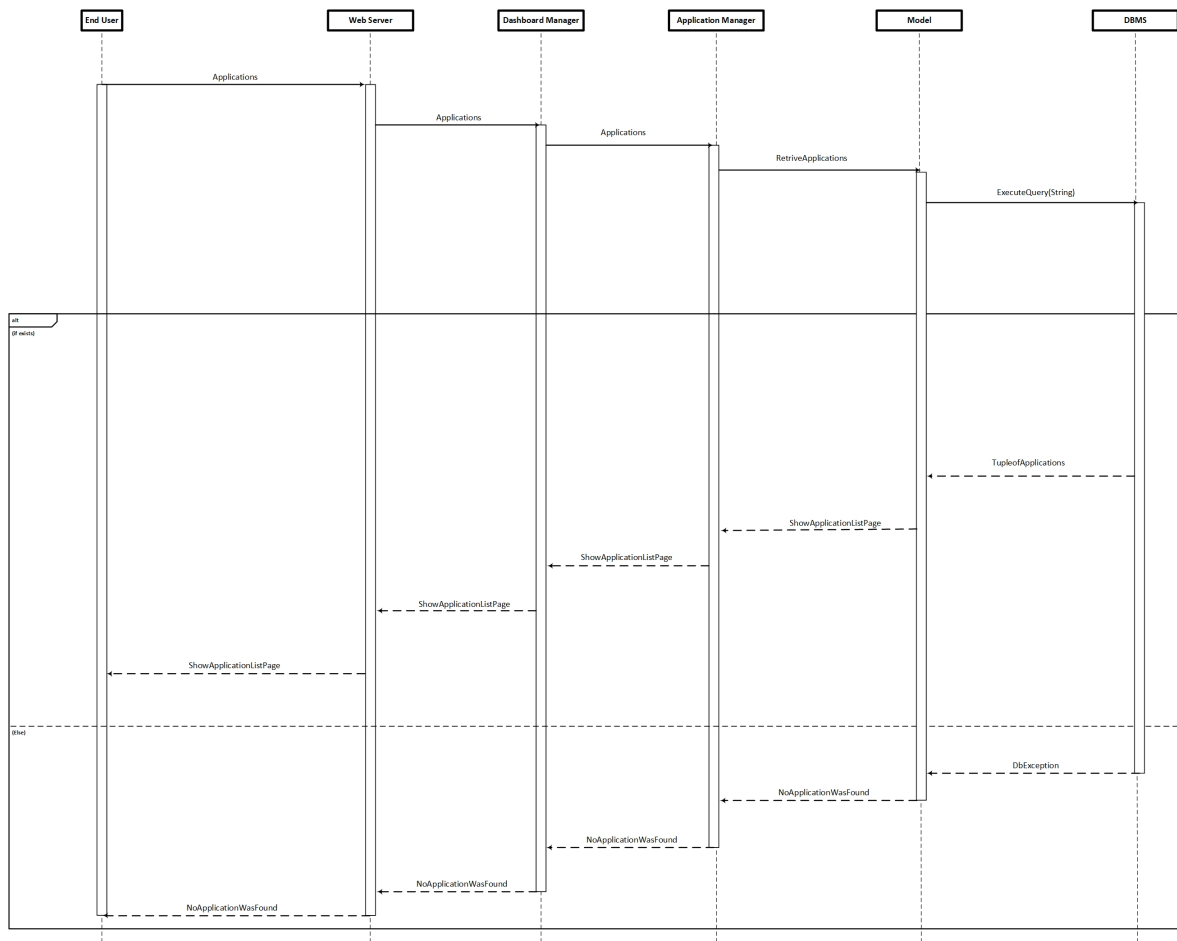


Figure 13: Track Application Runtime View

This diagram shows the Application Tracking process. Students/University log into their dashboard and request updates on their submitted applications. The DashboardManager forwards the request to the TrackingManager, which fetches the latest status from the database via the Model interface. The system displays the status, such as "Under Review" or "Shortlisted," to the student/university in real time.

2.4.9 Complaint Submission

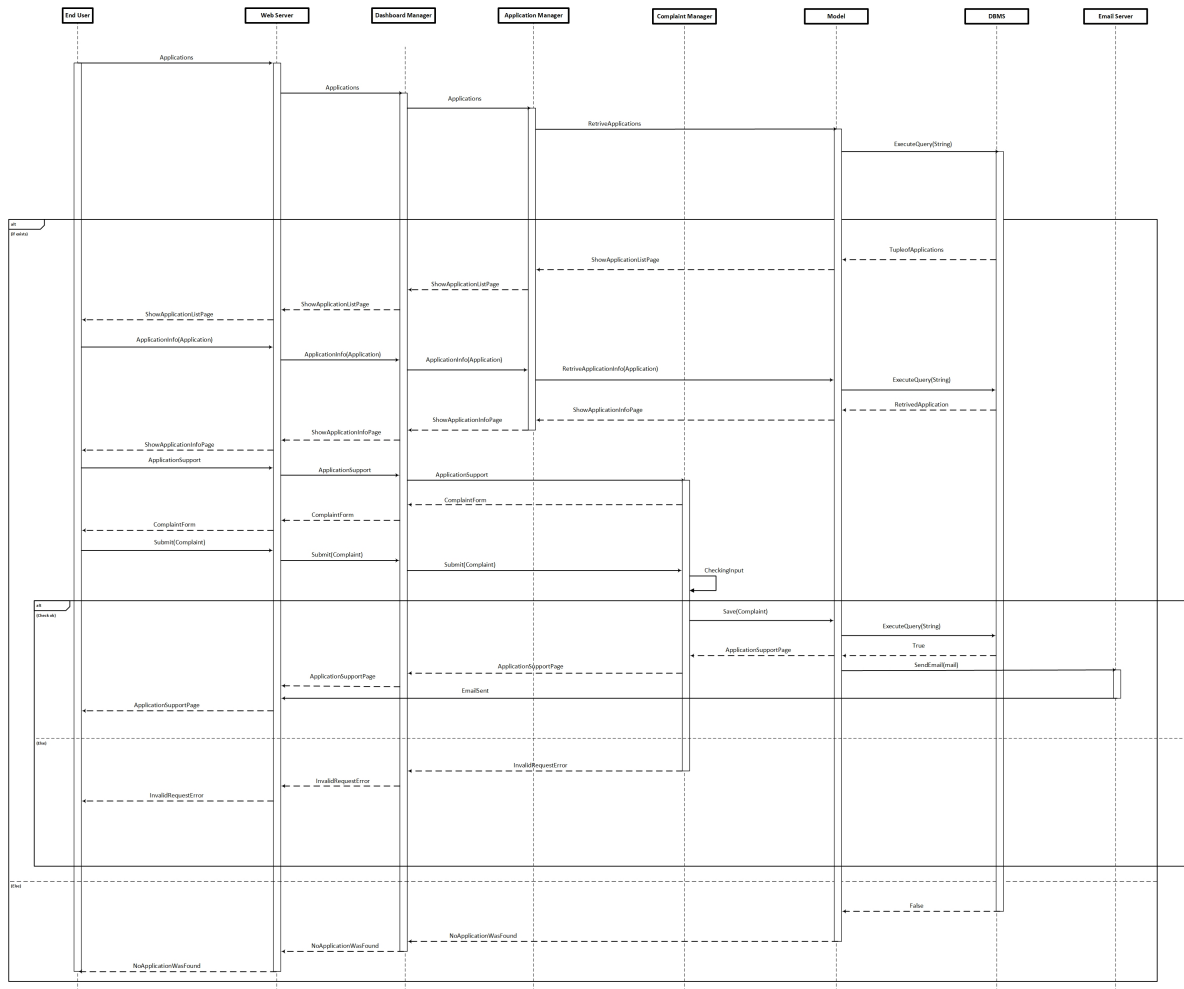


Figure 14: Complaint Submission Runtime View

This diagram represents the Complaint Submission process. A student navigates to the complaints section of their dashboard and fills out a form describing their issue. The form is sent to the Dashboard-Manager, which forwards it to the ComplaintManager. The ComplaintManager saves the complaint in the database via the Model interface and assigns it to the appropriate university. Notifications about the complaint status are sent to the student and university representatives.

2.4.10 Questionnaire Submission

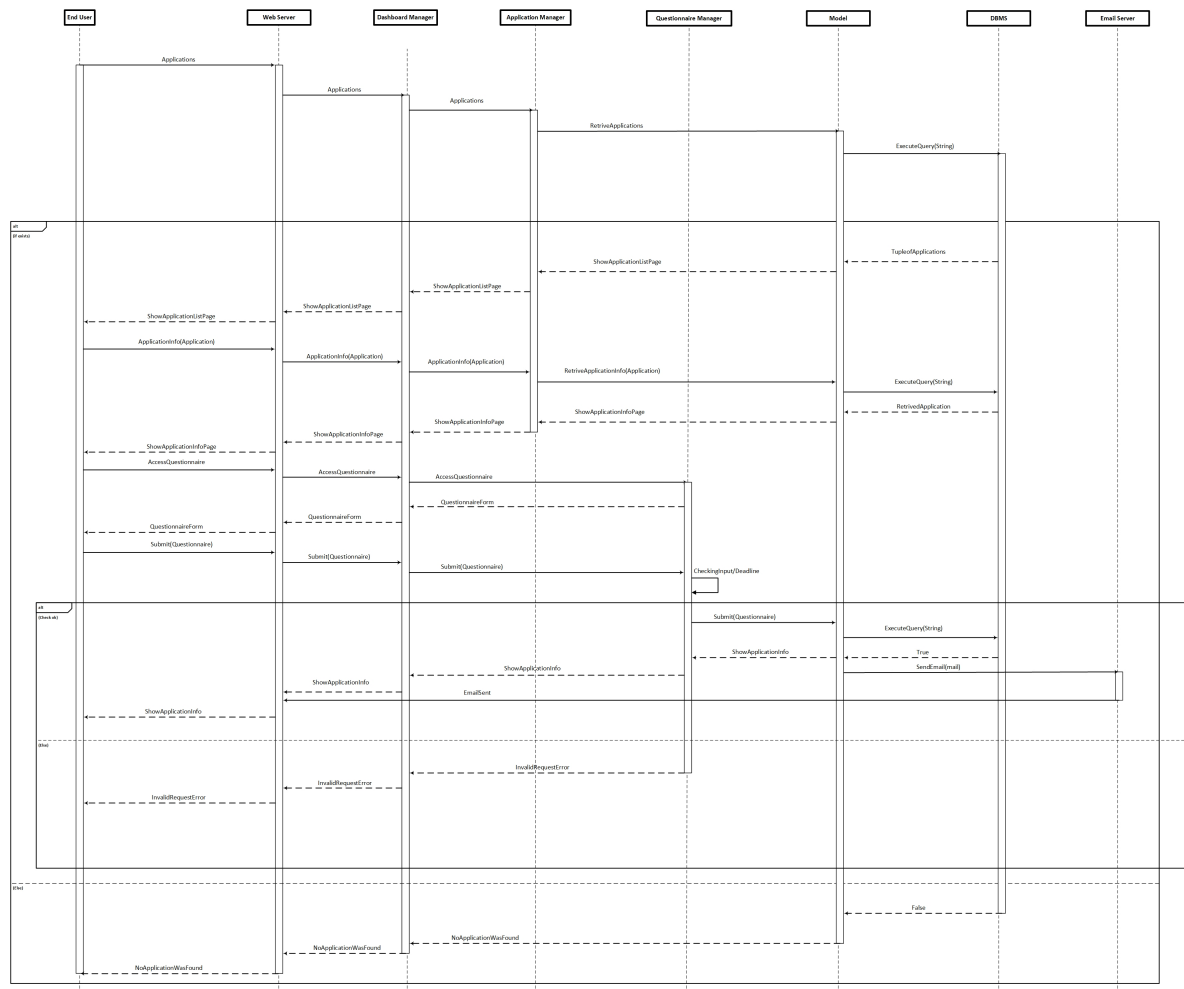


Figure 15: Questionnaire Submission Runtime View

This diagram illustrates the Questionnaire Submission process. Students access a questionnaire assigned to them as part of an internship application. They complete the questionnaire on the platform, and the responses are submitted via the DashboardManager to the QuestionnaireManager. The responses are saved in the database through the Model interface and linked to the respective application. A confirmation is displayed to the student.

2.4.11 Accept/Reject Offer

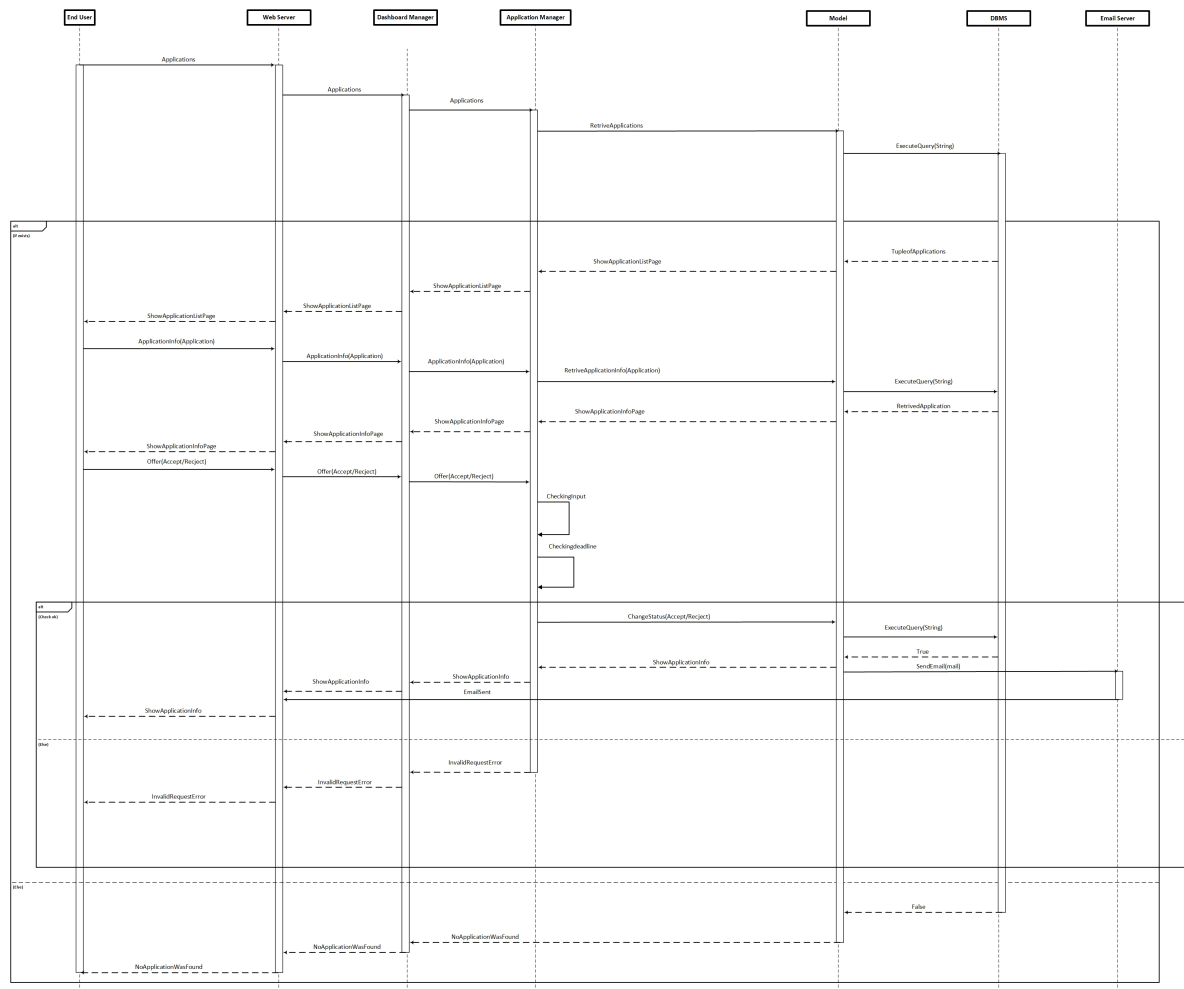


Figure 16: Accept/Reject Offer Runtime View

This diagram explains the Offer Decision process. A student logs into their dashboard and navigates to the offers section, where they see details of any internship offers extended to them. The student either accepts or rejects the offer. This decision is forwarded to the DashboardManager, which sends it to the InternshipManager. The InternshipManager updates the application status in the database through the Model interface. Notifications are then sent to both the student and the company via the NotificationManager to confirm the student's decision.

2.4.12 Meeting Attendance

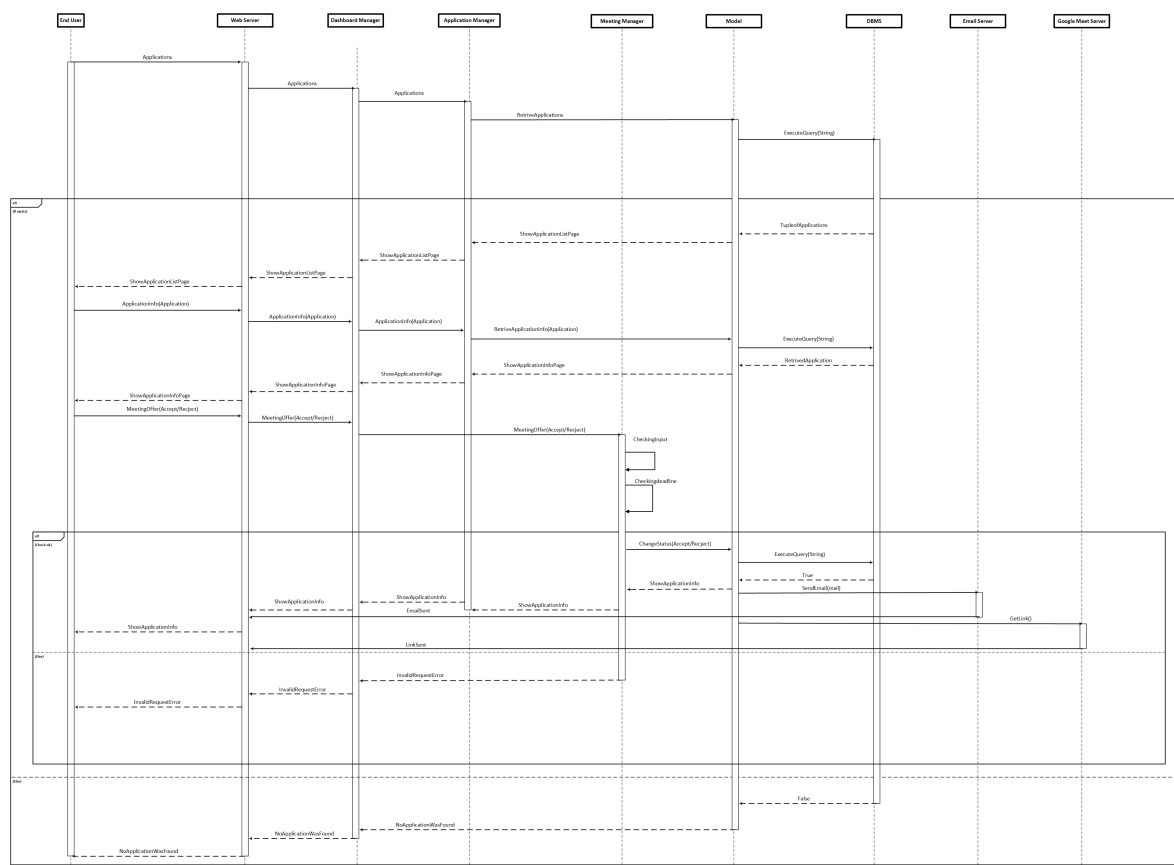


Figure 17: Accept/Reject Meeting Attendance Offer Runtime View

This diagram illustrates the Meeting Scheduling and Attendance process. A company schedules a meeting with a student, and the MeetingManager integrates with Google Meet and Google Calendar via their respective APIs to set the meeting details. The student receives a notification through the NotificationManager with a link to the meeting. After attending the meeting, the attendance record is logged in the database by the MeetingManager via the Model interface. Both the company and student dashboards are updated to reflect the meeting status.

2.4.13 Internship Posting

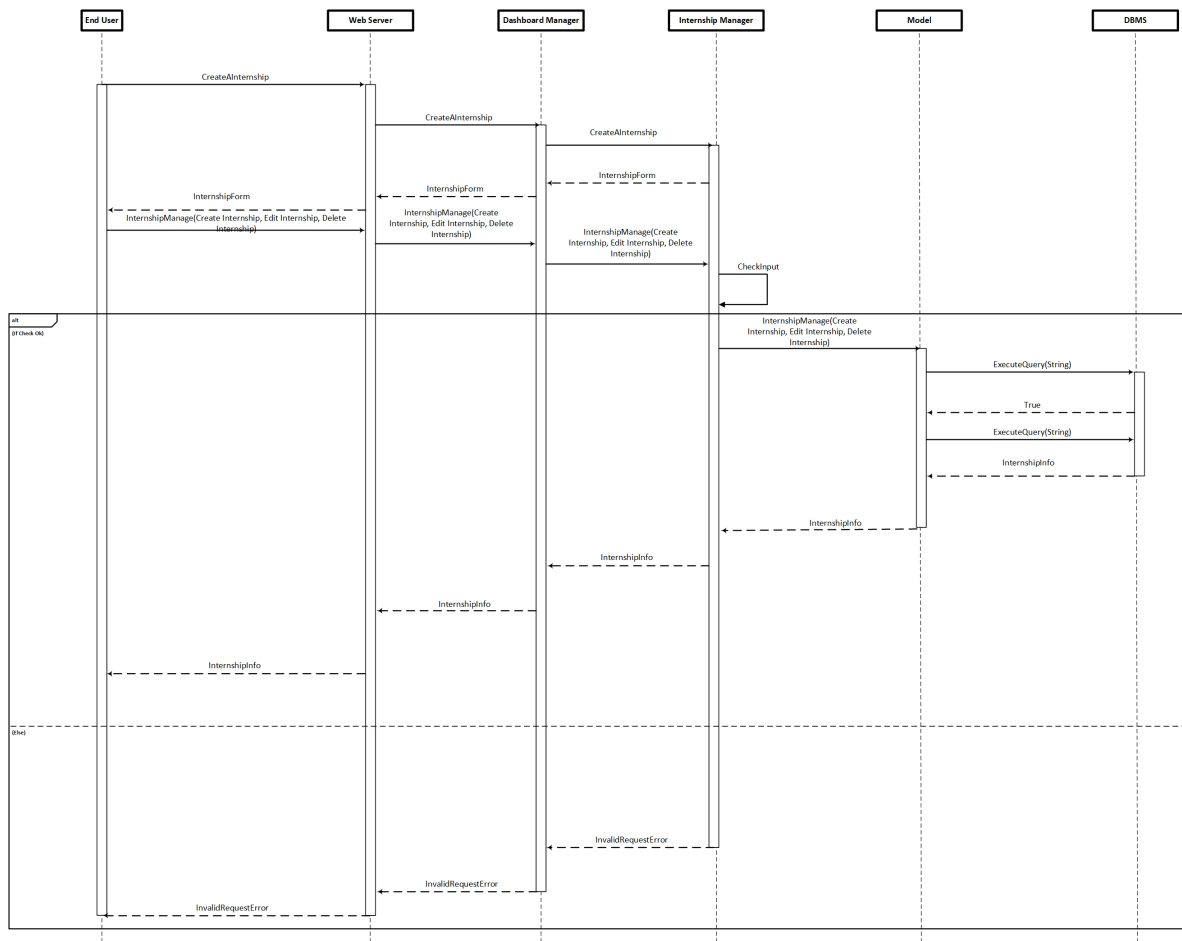


Figure 18: Internship Posting Runtime View

This diagram describes the Internship Posting process. A company representative logs into their dashboard and navigates to the internship posting form. They fill in details such as job title, description, requirements, and application deadlines. The form is submitted to the DashboardManager, which forwards the data to the InternshipManager. After validating the data, the InternshipManager saves it in the database through the Model interface. The posted internship is then made visible to students browsing the platform.

2.4.14 View Applicants

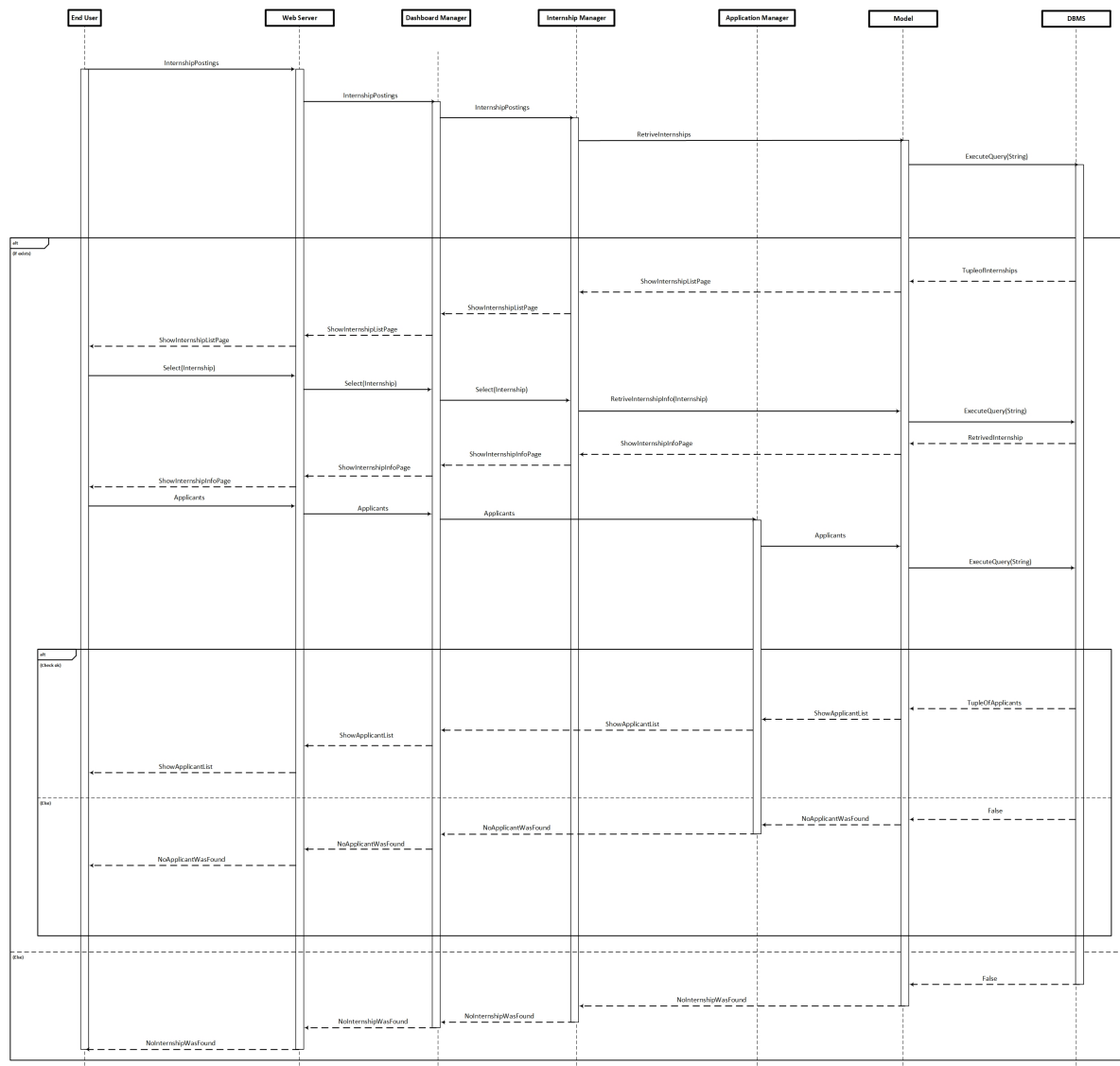


Figure 19: View Applicant Runtime View

This diagram represents the Applicant Viewing process. A company representative selects an internship on their dashboard and requests to view the list of applicants. The request is sent to the DashboardManager, which forwards it to the ApplicationManager. The ApplicationManager retrieves applicant details from the database using the Model interface and returns the results to the company's dashboard for review.

2.4.15 Shortlist Applicants

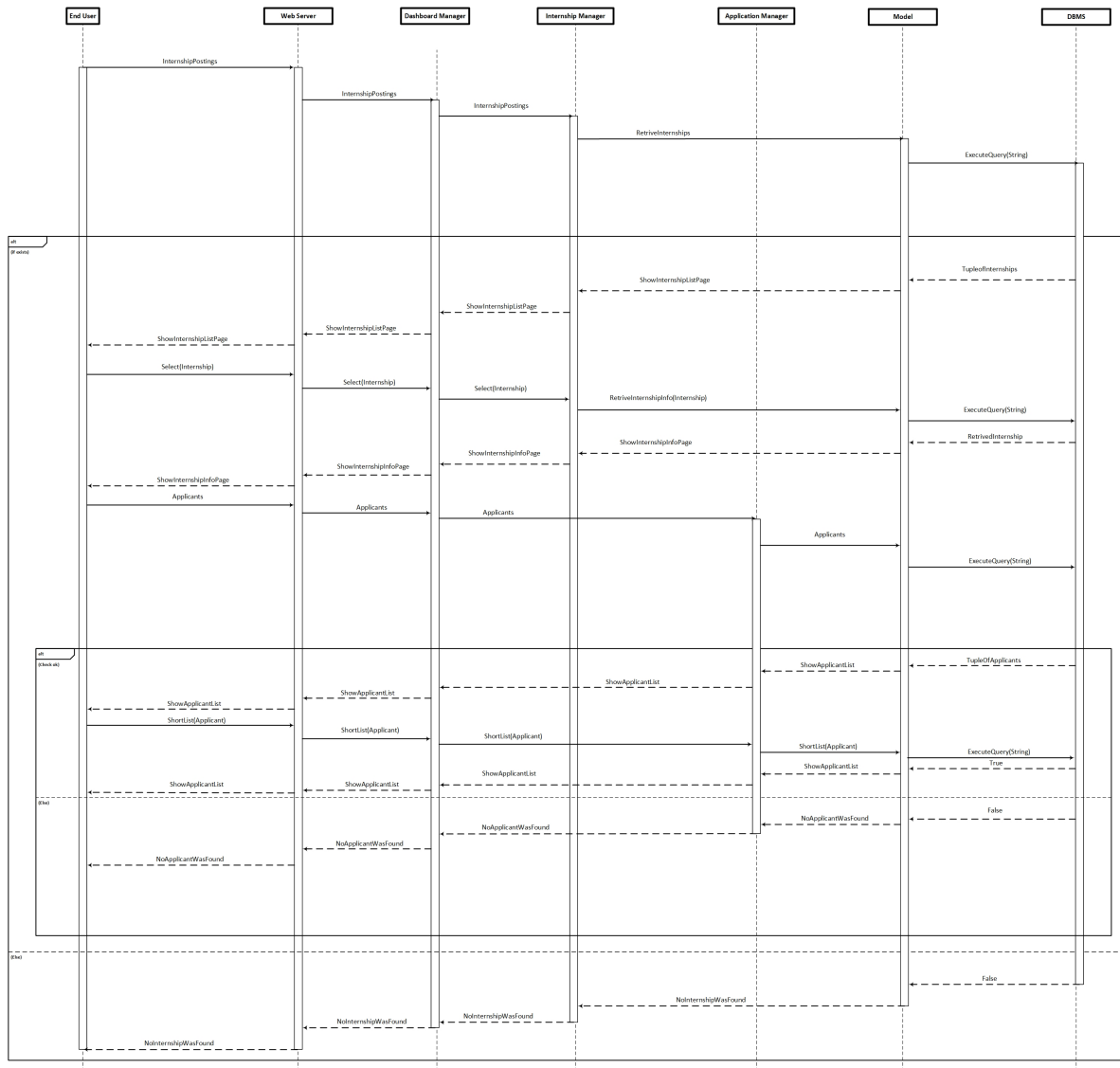


Figure 20: Shortlist Applicant Runtime View

This diagram explains the Shortlisting Process. A company representative reviews the list of applicants for a specific internship and selects candidates to be shortlisted. This action is sent via the Dashboard-Manager to the ApplicationManager, which updates the applicant statuses in the database through the Model interface. Notifications are sent to shortlisted candidates via the NotificationManager to inform them of their updated status.

2.4.16 Schedule Meeting

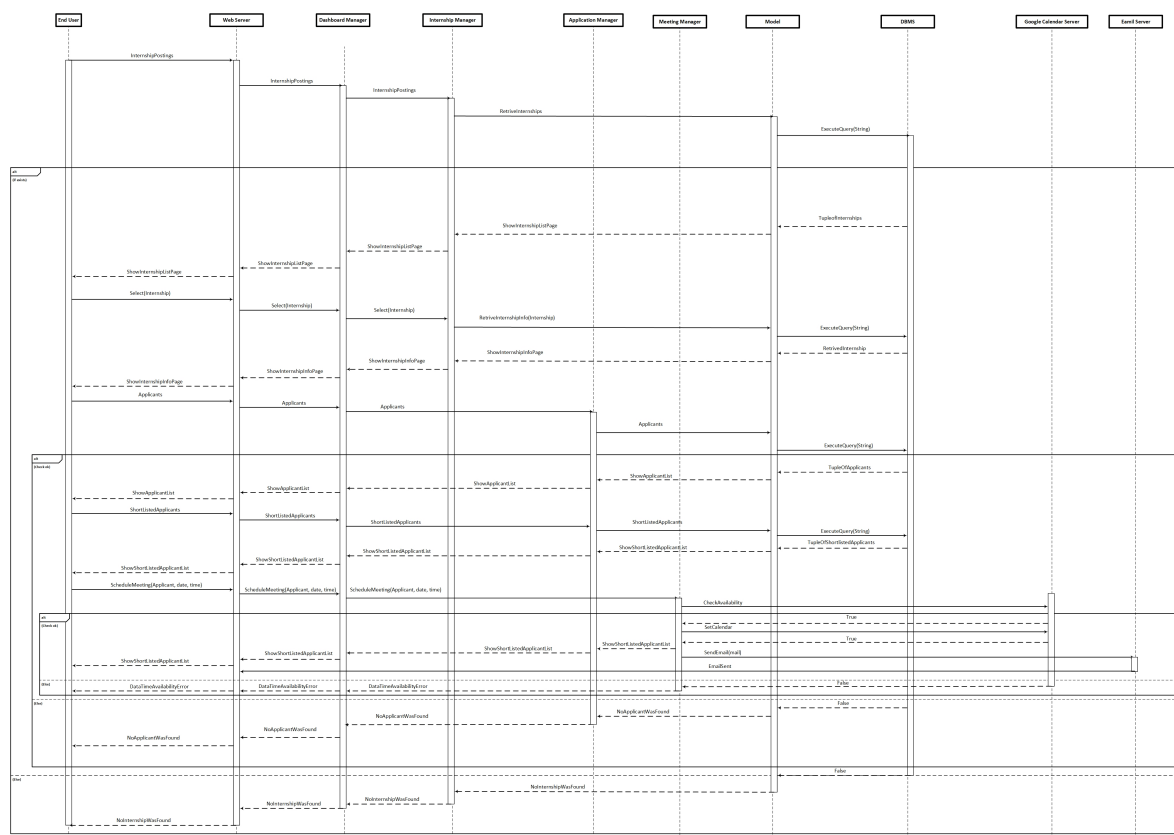


Figure 21: Schedule Meeting Runtime View

This diagram illustrates the Meeting Scheduling process. After shortlisting candidates, a company representative initiates a meeting request for interviews. The DashboardManager forwards the request to the MeetingManager, which uses Google Calendar and Google Meet APIs to create the meeting. Notifications are then sent to the candidates through the NotificationManager, providing details such as the meeting time and link.

2.4.17 Manage Questionnaire

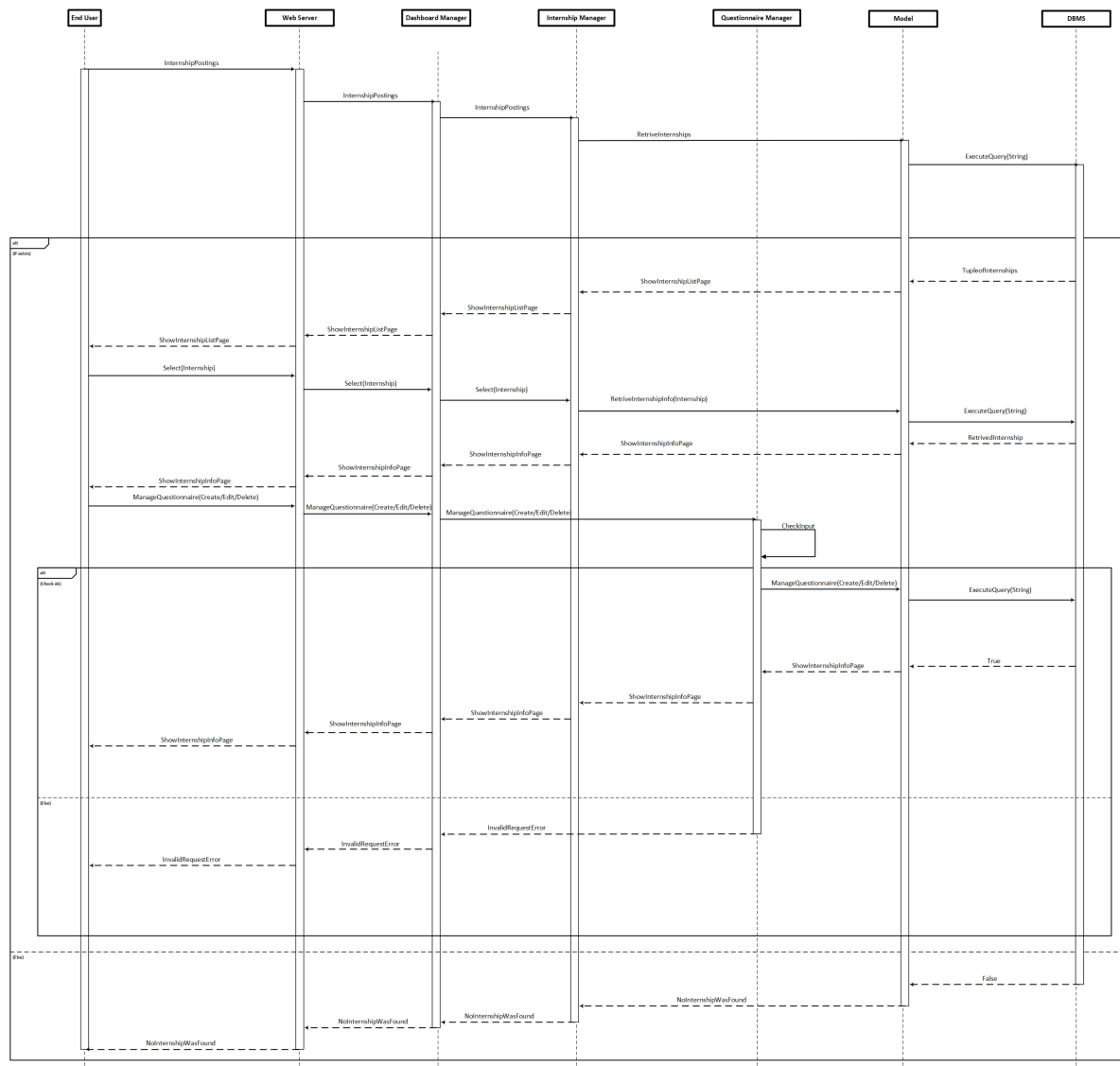


Figure 22: Manage Questionnaire Runtime View

This diagram describes the Questionnaire Management process. A company representative creates a new questionnaire for assessing candidates. The questionnaire details are submitted via the Dashboard-Manager to the QuestionnaireManager, which validates and saves the data in the database through the Model interface. The questionnaire is then linked to relevant internships or applications. Notifications are sent to students via the NotificationManager to inform them about the new assignment.

2.4.18 Send Offer

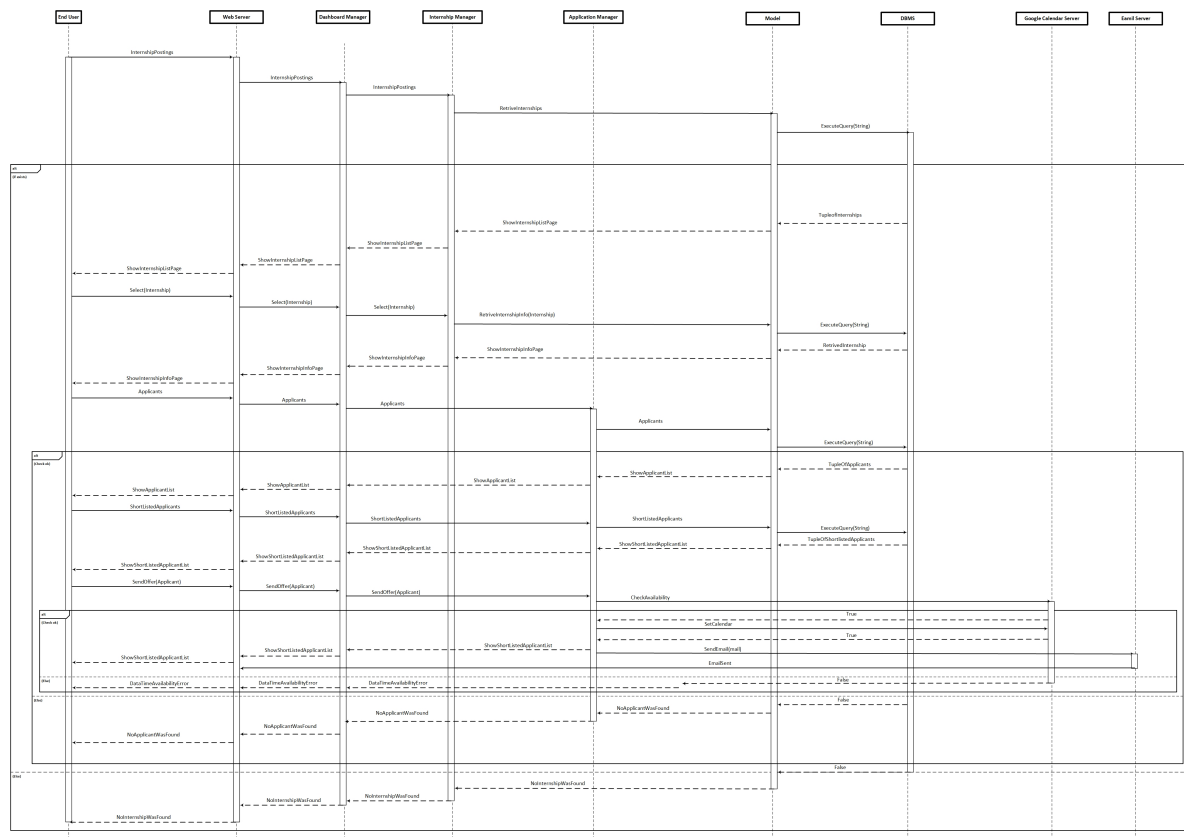


Figure 23: Send Offer Runtime View

This diagram represents the Offer Dispatching process. After completing interviews, a company selects candidates to receive internship offers. The offer details are sent via the `DashboardManager` to the `InternshipManager`, which saves the offer data in the database through the `Model` interface. Notifications are sent to the selected candidates via the `NotificationManager` to inform them of the offer.

2.4.19 Update Applicant Status

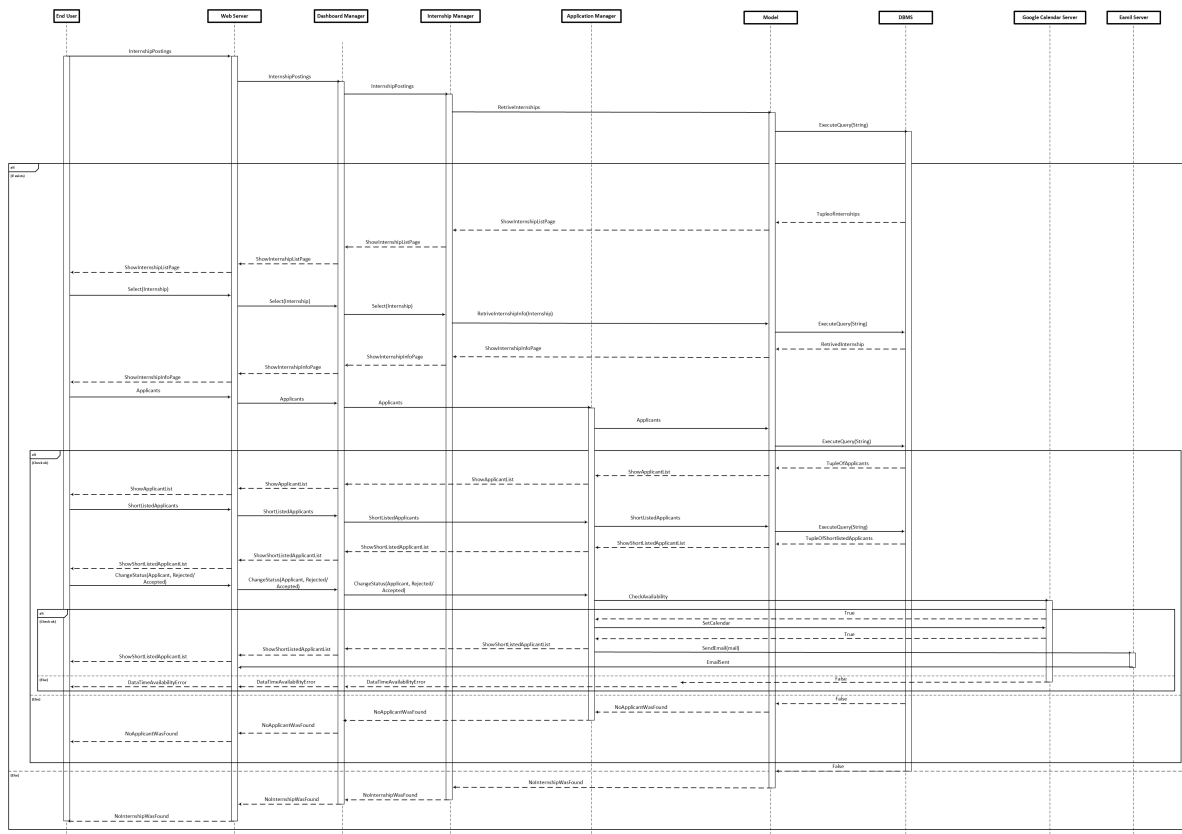


Figure 24: Update Applicant Status Runtime View

This diagram explains the Status Update process. A company representative updates the status of applicants (e.g., shortlisted, rejected, or hired) via the dashboard. The action is sent to the DashboardManager, which forwards it to the ApplicationManager. The ApplicationManager updates the database with the new statuses via the Model interface. Notifications are sent to the affected applicants via the NotificationManager.

2.4.20 Monitor Internships

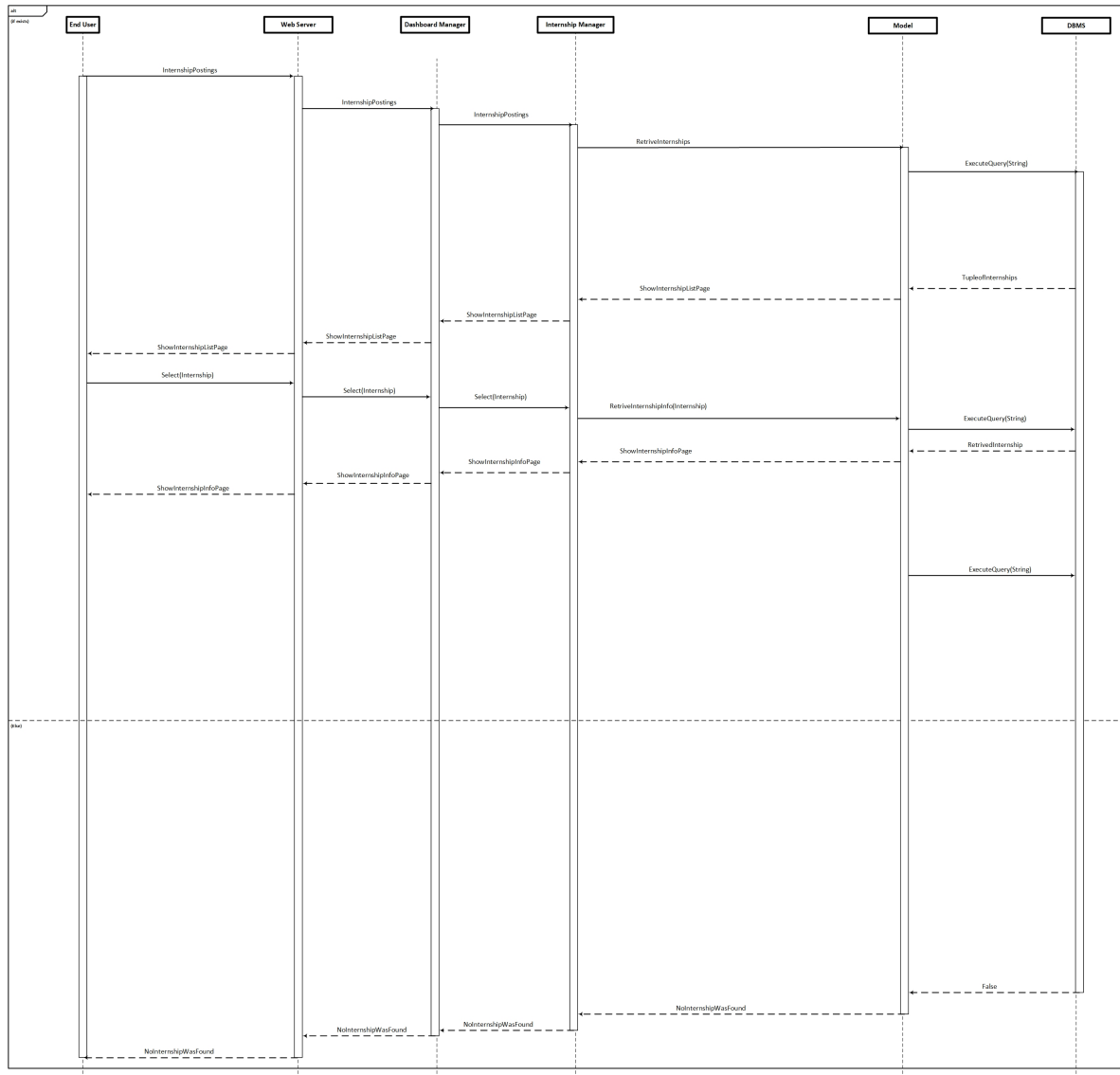


Figure 25: Monitor Internships Runtime View

This diagram illustrates the Internship Monitoring process. University representatives access their dashboard and request to monitor the progress of their students' internships. The DashboardManager forwards the request to the TrackingManager, which retrieves the necessary data from the database via the Model interface. The information is then displayed on the university dashboard, showing details such as student performance and application statuses.

2.5 Component Interfaces

This section lists all the methods that each component interface provides to the other components.

Registration

- CreateAnAccount()
- Register(*string* name, *string* surname, *string* nickname, *string* university affiliation, Email, password, sign-up as student tick)

- SaveNewUser(*string* name, *string* surname, *string* nickname, *string* university affiliation, Email, password, sign-up as student tick)
- Register(*string* organization name, *string* organization department, *string* representative name, *string* representative surname, *string* representative position, company email and password, and sign-up as company tick)
- Register(*string* university name, *string* university department, *string* university representative name, *string* representative position, email, password, sign-up as university tick)

Login

- Login(Email, *string* password)

Manage Resume

- CreateAResume()
- Resume(*File* Resume, checkbox for delete)

Internship

- SearchInternship(*string* Keyword)
- CreateAInternship()
- Check()
- Select(Internship)

Application

- Apply(*File* Resume)
- Check(**File** Resume)
- CheckDeadline()
- RetriveApplications()
- ApplicationInfo(Application)
- RetriveApplicationInfo(Application)
- Offer(Applicant, Accept/Reject Checkbox)
- ChangeStatus(Applicant, Accpet/Reject)
- RetriveApplicants(Internship)
- Shortlist(Applicant, Internship)

Complaint

- ComplaintForm()
- Submit(Complaint)
- Check(Complaint)
- Save(Complaint)

Questionnaire

- QuestionnaireForm()
- CheckDealine()

- Check(Questionnaire)
- Submit(Questionnaire)
- Save(Questionnaire)

Meeting

- MeetingOffer(Accept/Reject checkbox)
- CheckDeadline()
- Check()
- ScheduleMeeting(Applicant, date, time)

Notification

- MeetingNotification(Applicant, date, time)
- RegistrationNotification(Applicant)
- ApplicationNotifcation(Applicant, Internship)
- QuestionnaireNotification(Applicant, Internship, Application)
- ComplaintNotification(Applicant, University, Internship, Company)

Model

- CheckUserCreditionals(User)
- SaveNewUser(*string* organization name, *string* organization department, *string* representative name, *string* representative surname, *string* representative position, company email and password, sign-up as company tick)
- SaveNewUser(*string* university name, *string* university department, *string* university representative name, *string* representative position, email, password, sign-up as university tick)
- CheckEmail(email)
- CheckPassword(password)
- SaveNewResume(*File* Resume, checkbox for delete)
- GetAllInternship(keyword)
- GetAllApplicants(Internship)
- CreateApplication(Application)
- GetAllApplicants(university)
- GetAllComplaint(University)
- ChangeStatus(Applicant, Accept/Reject)
- InternshipManage(...)
- GetAllShortListApplicant(Application/Internship)
- ManageQuestionnaire(...)
- RetriveInternshipInfo(Internship)
- RetriveApplicationInfor(Application)
- GetUser(email)

- GetInternship(id)

Dashboard

- contains all the interfaces belonging to other managers, apart from Notification Manager and Model

WebServer

- contains all the interfaces belonging to other managers, apart from Notification Manager and Model

Email

- SendEmail(mail)

Google Calendar

- CheckDateTimeAvailability(User, date, time)
- SetCalendar(User, date, time)

Google Meet

- Sendlink(date, time, email)

2.6 Selected Architectural Styles and Patterns

S&C will be created using a 3-Tier architecture, which divides applications into logical and physical computing tiers. By running each layer on its own infrastructure, the system becomes more flexible and scalable while reducing complexity. Separate teams of developers may create each tier independently, allowing for updates and scaling without affecting the others.

The system is modularized into three distinct tiers:

- **Presentation Tier:** this is the top-level tier, that directly interacts with the Users. Its goal will be to collect the Users' inputs and show them the outputs produced by the lower tiers.
- **Application Tier:** this is the middle level tier. The Application Tier processes the Users' requests that arrive from the Presentation Tier, perform the necessary computation, even retrieving or storing data on the Data Tier, and computes that data in order to complete specific tasks requested by the Users. This tier needs to handle several requests by different Users simultaneously, while guaranteeing the security and integrity of the stored data.
- **Data Tier:** this is the lowest level tier. The Data Tier stores, retrieves and manages data used or produced by the Application Tier by the DataBase System's APIs.

The system will function as a Client-Server architecture, with the Client serving as the front-end user interface and the Server as the backend platform for processing and displaying user requests. In a pure Client-Server design, the Server initiates the request and acts as a passive element, processing and responding to the Client's queries. In certain situations, including when delivering notifications or interacting with other tools, the server must do actions without the client's initial request, making it an active component with more event-driven behavior.

REST APIs, a collection of instructions frequently used in the context of online transactions, are used to carry out the client-server interactions. Utilizing a stateless server, the client is the component that must inform the server of the transaction's status. This is known as representational state transfer. This method makes the server more scalable and facilitates the handling of numerous requests pertaining to several states at once. The HTTP protocol is used to share data and encode it in the JSON format because the components communicate by sending a representation of a resource in a manner that the server can understand.

The Model-View-Controller pattern, a software design pattern that divides the program into three interconnected pieces, will be followed in the development of S&C:

- **Model:** contains the methods that manage the data. It provides methods for saving, retrieving and manipulating data from the database.
- **View:** the view is the part that concerns the visual representation of the data for the final user.
- **Controller:** acts as a connection point between the view and the model. The controller handles the users' interactions with the view and processes the operations.

Facade pattern

In this pattern, a component offers an interface that simplifies the use of another component. The GoogleMeet and GoogleCalendar components use it to adapt and simplify the use of external services.

2.7 Other Design Decisions

In this section are presented some of the design decisions made for the system to make it work as expected.

2.7.1 Availability

Load balancing and replication are concepts that have been introduced in the system to increase the availability and reliability of the system. In this way the system would be as fault-tolerant as possible regarding data management and service availability.

2.7.2 Scalability

Microservices architecture, at its core, is meticulously crafted to be both independently deployable and inherently scalable. Because each microservice can be deployed independently thanks to this design philosophy, changes and alterations may be made quickly without affecting the system as a whole. Furthermore, by effectively expanding individual services, microservices' scalability feature enables our system to adapt to growing needs with grace.

2.7.3 Notification Handling

Notification management plays a pivotal role in the S&C system, working in every stage of application, from initiating application by the student to finishing internship. Notifications in S&C are orchestrated to reach users upon login or, if a User is already logged in, immediately when they become available. This approach guarantees that Users are promptly informed about key events, such as change of status in application, meeting offering, shortlisted and etc.

2.7.4 Ease of Deployment

The use of microservices architecture offers a significant benefit in terms of deployment simplicity. This approach enables the autonomous implementation of modifications to certain services, enabling a smooth deployment without affecting the system as a whole. The microservices method eliminates the need to stop the entire system in the event of problems by facilitating quick identification, isolation, and rectification. In addition to speeding up update releases, this deployment flexibility improves the system's robustness and agility, facilitating effective maintenance and troubleshooting procedures.

2.7.5 Data Storage

We have selected a unified database management system (DBMS) that contains data on users, internship postings, application, questionnaires, and complaints in order to improve operational efficiency and streamline data management. This method uses the interconnected links between these components to decrease the time and complexity involved in retrieving and updating data.

3 User Interface Design

3.1 Overview

3.2 Header

3.3 User Interfaces

4 Requirements Traceability

5 Implementation, Integration and Test Plan

6 Effort Spent

References

List of Tables

1	Abbreviations	3
.....		

List of Figures

1	Three tier architecture	4
2	System Architecture	5
3	High Level Component Diagram	5
4	Low Level Component Diagram	7
5	Deployment View Diagram	9
6	Student Registration Runtime View	11
7	Company Registration Runtime View	12
8	University Registration Runtime View	13
9	Login Runtime View	14
10	Manage Resume Runtime View	15
11	View Internship Runtime View	16
12	Sending Application Runtime View	17
13	Track Application Runtime View	18
14	Complaint Submission Runtime View	19
15	Questionnaire Submission Runtime View	20
16	Accept/Reject Offer Runtime View	21
17	Accept/Reject Meeting Attendance Offer Runtime View	22
18	Internship Posting Runtime View	23
19	View Applicant Runtime View	24
20	Shortlist Applicant Runtime View	25
21	Schedule Meeting Runtime View	26
22	Manage Questionnaire Runtime View	27
23	Send Offer Runtime View	28
24	Update Applicant Status Runtime View	29
25	Monitor Internships Runtime View	30